

Redrob Ranker v2.0 — "Council of Nine"

Technical Report: Approach, Architecture, Rationale & Mechanics

An intelligent candidate **ranking engine** for the Redrob *Intelligent Candidate Discovery & Ranking Challenge*. It ranks a 100,000-profile pool against a fixed Senior-AI-Engineer job description, CPU-only, fully offline, deterministically, in ~25 seconds, and emits a spec-compliant top-100 CSV with grounded per-candidate reasoning.

This report is grounded entirely in the repository source. Every claim references a real file, function, variable, or config value. Where the code leaves something implicit, that is stated rather than guessed.

Repository under analysis: `redrob-ranker-v2/` **Single reproduce command:** `python rank.py --candidates ./candidates.jsonl --out ./submission.csv`

The problem being solved (context for everything below)

- **Input:** `candidates.jsonl` — 100,000 candidate profiles (one JSON object per line). Each has `profile`, `career_history[]`, `education[]`, `skills[]`, `certifications[]`, `languages[]`, and a `redrob_signals` block of behavioural telemetry.
- **Target:** a single fixed job description (the "Senior AI Engineer — Founding Team" role), interpreted statically in `src/jd_intent.json`.
- **Output:** exactly the top 100 candidates as a CSV with header `candidate_id,rank,score,reasoning` (enforced by `validate_submission.py`).
- **Official scoring metric:** $0.50 \cdot \text{NDCG@10} + 0.30 \cdot \text{NDCG@50} + 0.15 \cdot \text{MAP} + 0.05 \cdot \text{P@10}$ (cited in `README.md` and `src/score.py`). Ordering matters as much as set membership.
- **Hard constraints:** CPU-only, "Network Off", ≤16 GB RAM, ≤5-minute ranking budget, deterministic and reproducible. ~80 "honeypot" profiles are planted; flagging more than 10% of the pool risks disqualification.

These constraints shape every design decision in the sections that follow.

1. Approach

1.1 Overall strategy: retrieve, then judge, then calibrate

The system treats ranking as a classic **two-stage information-retrieval problem**, wrapped in an interpretable scoring layer that encodes the job description's *priorities* rather than its keywords:

1. **Recall stage (retrieval).** A hybrid lexical + dense retriever scores all 100K profiles against the JD query and keeps a generous shortlist of `SHORTLIST_SIZE = 4000` candidates (`src/retrieve.py`, `src/config.py`). The goal here is *recall* — never drop a hidden gem.

2. **Precision stage (the "Council of Nine")**. Each shortlisted candidate is scored by nine independent, interpretable sub-scorers (`src/council.py`), six of which are fused additively and three of which act as multiplicative gates. This is where the strategic judgement lives.
3. **Order-sharpening + calibration**. The head of the list is re-ranked (`src/rerank.py`) and the whole list is placed into grounded relevance bands with disjoint score sub-ranges (`src/score.py::finalize_ranking`), producing the globally non-increasing, tie-free ordering the metric and validator require.

The guiding philosophy, stated in `README.md` and `submission_metadata.yaml`, is "**read profiles instead of counting keywords**": career evidence and role identity dominate the score; the self-declared skills list is deliberately down-weighted and gated by a verification trust factor; a conservative integrity layer floors impossible honeypots; and a bounded behavioural modifier separates engaged candidates from dormant ones.

1.2 ML approach: pointwise core, with pairwise/listwise sharpening

There are no relevance labels in the dataset, so this is **not** a supervised learning-to-rank model trained on graded judgements. Instead it is an **unsupervised, heuristic ensemble** that blends all three LTR paradigms at different stages:

Paradigm	Where it is used	Why
Pointwise	The Council-of-Nine composite in <code>council.deliberate</code> + the gating/availability multipliers in <code>score.score_pool</code> produce one independent score per candidate.	No labels exist to train a pairwise/listwise loss; a pointwise interpretable score is auditable (an EU AI Act requirement) and fully deterministic.
Pairwise (neural)	Stage <code>[3b]</code> cross-encoder re-rank of the top- <code>RERANK_SIZE = 200</code> head (<code>src/rerank.py</code> , <code>cross-encoder/ms-marco-MiniLM-L-6-v2</code>). A cross-encoder implicitly learns pairwise relevance.	Sharpens NDCG@10 ordering at the very top, where the metric weights most (0.50).
Listwise	Stage <code>[9b]</code> 3-band relevance gate + intra-band calibration (<code>score.finalize_ranking</code> , <code>score._assign_banded_scores</code>).	Guarantees higher-relevance candidates <i>always</i> outrank lower ones — the property MAP and P@10 reward — while preserving fine-grained within-band order for NDCG .

So the chosen approach is **pointwise scoring as the backbone, with a pairwise neural re-rank on the head and a listwise band calibration over the whole list**. This was chosen because (a) labels are absent, (b) explainability is mandatory, and (c) the composite metric rewards both correct *sets* and correct *order*.

1.3 Text representation: hybrid TF-IDF (lexical) + LSA (dense), RRF-fused

The default, always-available representation (in `src/retrieve.py::HybridRetriever`) is:

- **Lexical signal — TF-IDF**. `TfidfVectorizer(max_features=50000, ngram_range=(1,2), min_df=2, sublinear_tf=True)` . Uni- and bi-grams capture phrases like "vector search" and "learning to rank". This is the README's stand-in for a SPLADE-style learned-sparse signal.
- **Dense signal — LSA**. `TruncatedSVD(n_components=DENSE_DIM=256, n_iter=4, random_state=13)` applied to the TF-IDF matrix, then L2-normalised. SVD is *fit on a 40,000-row sample* (`SVD_FIT_SAMPLE`) and used to *transform all rows* — a large speed win. This is the stand-in for a neural sentence-

embedding model.

- **Fusion — Reciprocal Rank Fusion.** Lexical and dense rankings are merged with RRF (`RRF_K = 60`) via a vectorised inverse-rank scatter in `HybridRetriever.retrieve` , then the top 4000 indices form the shortlist.

Why this representation, and not neural embeddings by default? Because the ranking step must run offline, on CPU, deterministically, and inside 5 minutes, with no model weights to download or commit. TF-IDF + LSA needs zero downloads and is bit-for-bit reproducible (`RANDOM_SEED = 13`). Real embeddings are supported as a **pluggable, default-on-with-fallback upgrade**: `src/retrieve.py::STRetriever` uses `sentence-transformers` (`BAAI/bge-small-en-v1.5` , 384-dim) when installed and cached, and `build_retriever` falls back to LSA instantly if the model or its weights are unavailable (`requirements-embeddings.txt` documents the opt-in).

1.4 Ranking model(s): the Council of Nine + two-stage re-rank + bands

The scoring model is an **ensemble of nine interpretable sub-scorers** ("Navaratna"), each mapping a human-judgement principle to a concrete, measurable feature (`src/council.py`):

#	Scorer (function)	Type	Measures
1	<code>semantic_seer</code>	additive (w=0.16)	dense JD↔profile semantic similarity
2	<code>name_rectifier</code>	additive (w=0.20)	does the title match reality? (anti-stuffer)
3	<code>evidence_scout</code>	additive (w=0.22)	demonstrated "built / shipped a system" evidence
4	<code>mask_piercer</code>	additive (w=0.14)	verified skill-trust (endorsements × duration × assessment)
5	<code>path_reader</code>	additive (w=0.12)	experience-band fit + tenure stability (anti job-hopper)
6	<code>terrain_master</code>	additive (w=0.16)	product-vs-services company + NLP/IR domain proximity
7	<code>neti_net</code>	multiplier (floor 0.40)	negative screen (services-only, off-domain, stuffer)
8	<code>integrity.check</code>	gate (0 or 1× + soft)	plausibility / honeypot detection
9	<code>availability_oracle</code>	multiplier [0.55, 1.10]	behavioural availability (recency, responsiveness)

Plus a separate `disqualifier_screen` multiplier (floor 0.20) for the JD's explicit "we will not move forward" cases. The six additive scorers are fused with the weights in `config.COUNCIL_WEIGHTS` (normalised at runtime), and the gates are applied multiplicatively in `score.score_pool` :

```
final_fit = council_core × integrity × neg_mult × disqualifier_mult
final      = final_fit × availability_mult + soft_nudges (bounded additive bonuses)
```

On top of this pointwise score sit **Stage [3b]** (neural cross-encoder head re-rank, with a deterministic feature fallback) and **Stage [9b]** (3-band relevance gate + tier calibration). These specific models were chosen because each sub-scorer is independently explainable and tunable (every weight lives in `src/config.py` and is written to an audit trail), which is exactly what a high-risk hiring system needs — while the neural re-rank and bands recover the order-sensitivity that a flat weighted sum would lose.

1.5 The metrics being optimised

The submission is graded on a weighted blend (`README.md` , `src/score.py` , locked in by `tests/test_metrics.py`):

Metric	Weight	Meaning	How the design targets it
NDCG@10	0.50	Normalised Discounted Cumulative Gain over the top 10 — rewards putting the <i>most</i> relevant candidates <i>highest</i> , with a logarithmic position discount.	The cross-encoder head re-rank (<code>rerank.py</code>) sharpens the top-200 ordering; continuous within-band scores keep fine-grained position sensitivity.
NDCG@50	0.30	Same as above but over the top 50.	Same mechanisms; the 4000-deep shortlist guarantees the true top-50 are present to be ordered.
MAP	0.15	Mean Average Precision — rewards ranking <i>all</i> relevant candidates above <i>all</i> irrelevant ones.	The 3-band gate (<code>relevance_band</code>) forces STRONG > STANDARD > WEAK with disjoint score ranges, so a higher-relevance profile never sits below a lower one.
P@10	0.05	Precision@10 — the fraction of the top 10 that are genuinely relevant.	A band-0 (unqualified) profile is structurally barred from the top 10 when qualified candidates exist (<code>tests/test_metrics.py::test_p_at_10_floor_no_unqualified_in_top10</code>).

The key insight encoded throughout `score.py` is that **order matters as much as set membership**, which is why a flat weighted sum is deliberately *not* the final word — bands and a head re-rank are layered on top.

2. What Was Built

The repository is a complete product, not just a scoring script: a reproducible ranking core (`src/`), two entrypoints (`rank.py` , `precompute.py`), an agentic wrapper (`agents/`), a Streamlit sandbox (`app.py`), a FastAPI + Next.js dashboard (`api/` , `web/`), a compliance trail (`compliance/`), and a test suite (`tests/`).

2.1 Entrypoint & pipeline scripts (repo root)

Script	What it does	In	Out
<code>rank.py</code>	The single online entrypoint. Forces Hugging Face offline mode, loads the JD, streams candidates, loads (or refits) the retriever, runs the full Council pipeline, writes the audit trail, and emits the CSV.	<code>--candidates candidates.jsonl</code> , <code>--out submission.csv</code>	top-100 <code>submission.csv</code> + a <code>compliance/audit_trail/*.json</code>
<code>precompute.py</code>	The offline index builder (the heavy half of the two-phase design). Builds documents, fits the hybrid retriever, replays retrieval for the fixed JD query, and freezes the result.	<code>--candidates</code> , <code>--backend {auto,st,lsa}</code> , <code>--gzip</code> , <code>--warm-reranker</code>	<code>artifacts/retriever.pkl.gz</code> , <code>artifacts/candidate_ids.pkl.gz</code>

Script	What it does	In	Out
<code>validate_submission.py</code>	The official challenge validator, vendored in. Checks <code>.csv</code> extension, exact header, exactly 100 rows, <code>CAND_[0-9]{7}</code> IDs, unique ranks 1–100, non-increasing score by rank, and candidate_id-ascending tie-breaks.	a submission CSV	"Submission is valid." or a list of errors
<code>app.py</code>	A Streamlit sandbox (the mandatory \$10.5 demo). Pick a role, upload any-size pool, rank the whole pool, page the leaderboard, and export a spec CSV or detailed Excel.	uploaded JSON/JSONL	on-screen leaderboard + downloadable CSV/XLSX
<code>onnx_optimize.py</code>	Optional upgrade path: export a sentence-transformers/Qwen model to ONNX and apply INT8 dynamic quantization for a 2–3× CPU speedup. Not required by <code>rank.py</code> .	<code>--model</code> , <code>--quantize int8</code>	<code>artifacts/onnx/</code>

2.2 The ranking core — `src/` (every module)

Module	Stage	What it does	Input → Output
<code>config.py</code>	—	Central, auditable configuration: shortlist/rerank sizes, TF-IDF/SVD knobs, <code>COUNCIL_WEIGHTS</code> , availability bounds, disqualifier multipliers, honeypot thresholds, band ranges, <code>RANDOM_SEED=13</code> , <code>TOP_N=100</code> .	constants imported everywhere
<code>jd_intention.json</code>	[1]	Static, structured interpretation of the job description (so the ranking step needs no LLM/network). Holds <code>query_text</code> , <code>must_have_capabilities</code> , <code>nice_to_have</code> , positive/negative titles, evidence verbs/domain nouns, product/services lists, disqualifier vocabularies, preferred locations, experience band.	the JD object
<code>load.py</code>	[0a]	Streams the JSONL pool with <code>orjson</code> if available (else <code>stdlib json</code>); auto-detects a pretty-printed JSON array; tolerates malformed lines. <code>load_candidates(path, limit)</code> .	path → <code>list[dict]</code>
<code>features.py</code>	[0b]	The feature engine. <code>build_document(c)</code> builds the retrieval text (career <i>descriptions weighted</i> 2×). <code>compute_features(c, jd)</code> derives ~45 numeric/categorical features: tenure, product-ratio, title identity, domain-anchored evidence hits, verified skill-trust, off-domain/IR-NLP hits, recency, disqualifier signals, nice-to-have trust, eval-rigor hits, behavioural signals. <code>REFERENCE_DATE = 2026-06-13</code> fixes recency math.	<code>(candidate, jd) → dict</code> of features + a text document
<code>retriever.py</code>	[2]	<code>HybridRetriever</code> (TF-IDF + LSA + RRF), <code>STRetriever</code> (sentence-transformers dense backend), and <code>build_retriever(docs, backend)</code> which resolves the backend safely with automatic LSA fallback and a reduced "fast-fit" mode for large cold pools.	docs + JD query → <code>(shortlist_idx, dense_sim, lexical_sim)</code>
<code>precomputed.py</code>	[2]	<code>PrecomputedRetriever</code> — a numpy-only frozen index that <i>replays</i> the offline retrieval result (shortlist + similarities) for the fixed JD query. Deliberately sklearn-free so unpickling is instant.	frozen arrays → same <code>(shortlist, dense_sim, lexical_sim)</code>

Module	Stage	What it does	Input → Output
<code>council.py</code>	[3]	The Council of Nine. Functions <code>semantic_seer</code> , <code>name_rectifier</code> , <code>evidence_scout</code> , <code>mask_piercer</code> , <code>path_reader</code> , <code>terrain_master</code> , <code>neti_net</code> , <code>availability_oracle</code> , <code>disqualifier_screen</code> , and <code>deliberate(f, sem_sim)</code> which fuses the six additive scorers and returns parts, core, multipliers, and rationale fragments.	<code>(features, sem_sim)</code> → decision dict
<code>integrity.py</code>	[6]	The Integrity Warden. <code>check(c)</code> applies four HARD impossibility rules (≥3 expert-with-0-months skills; tenure exceeding its own date span; self-contradictory timeline; skill older than the technology via <code>TECH_FIRST_YEAR</code>) plus soft penalties. Conservative by design.	candidate → <code>(integrity_score, is_honeypot, reasons)</code>
<code>rerank.py</code>	[3b]	Two-stage head re-rank. <code>rerank(jd, head)</code> tries a CPU cross-encoder (<code>_cross_encoder_scores</code> , offline + time-guarded) and falls back to a deterministic <code>_feature_scores</code> re-rank. Default backend is <code>feature</code> .	JD + top-200 head → one relevance score per head record
<code>score.py</code>	[3] → [9]	The orchestrator. <code>score_pool</code> retrieves, computes features, gates honeypots, fuses the council, applies multipliers + <code>_soft_nudge</code> bonuses. <code>relevance_band</code> assigns 2/1/0. <code>finalize_ranking</code> does the head re-rank, band sort, and <code>_assign_banded_scores</code> calibration. <code>load_jd</code> loads the intent.	candidates + JD → 100 ranked records + stats
<code>reasoning.py</code>	[10]	<code>generate(...)</code> writes a 1–2 sentence, fact-grounded, rank-calibrated justification per candidate. Names only verified skills (via <code>skills_verify</code>), acknowledges honest concerns, and varies phrasing deterministically by <code>candidate_id</code> (md5 seed).	candidate + features + decision + rank → reasoning string
<code>fairness.py</code>	[7]	<code>audit(pool, selected_idx)</code> computes representation and the disparate-impact ("4/5ths") ratio for <code>region</code> and <code>institution_tier</code> . Audits and logs by default rather than forcibly re-ranking; deliberately does not infer gender.	pool + selected indices → fairness report
<code>skills_verification.py</code>	[3 helper]	<code>verified_relevant_skills(c, jd, top)</code> returns JD-relevant skills backed by endorsements/usage (so reasoning never hallucinates). <code>certification_credibility(c)</code> scores certs from <code>KNOWN_ISSUERS</code> .	candidate + JD → verified skill names / cert score
<code>compliance.py</code>	[9 helper]	<code>write_audit(...)</code> writes an immutable, timestamped EU AI Act record per run: input fingerprint (SHA-256), config weights, honeypot counts by rule, timing, backend, and the fairness report.	run metadata → <code>compliance/audit_trail/audit*.json</code>
<code>roles.py</code>	—	A role catalogue for the dashboard. Overlays role-specific fields (query, must-haves, titles, domain, experience band) onto the base JD so the same Council code ranks for six different roles.	role name → JD-shaped intent dict

2.3 Agentic orchestration — `agents/`

A thin, deterministic coordination layer over the same `src/` modules (mirrors the multi-agent diagram in the design blueprint, Section 8):

- `orchestrator.py` — `Orchestrator.run(candidates)` sequences the pipeline as named agents: **JD Parser** → **Source** → **Evaluate** → **Verify** → **Explain** → **Compliance**, calling `load_jd`, `score_pool`, `fairness.audit`, and `compliance.write_audit`.
- `specialists.py` — single-responsibility wrappers: `JDParserAgent`, `SourceAgent` (retrieval), `EvaluateAgent` (council), `VerifyAgent` (integrity + verified skills), `ExplainAgent` (reasoning). In production each would be its own microservice with the same interface.

2.4 Service & UI layer — `api/` (FastAPI) and `web/` (Next.js)

The Python ranking engine is unchanged and exposed through a service for an enterprise-style dashboard (`WEB_README.md`):

api/ (FastAPI, port 8000): - `main.py` — the FastAPI app. Endpoints: `/api/roles`, `/api/rank`, `/api/stage` (chunked upload), `/api/status`, `/api/summary`, `/api/leaderboard`, `/api/candidate/{id}`, `/api/analytics`, `/api/compliance`, `/api/honeypots`, `/api/job-intent`, `/api/export`, plus Supabase task/shortlist routes and `/api/nextai/chat`. - `ranker.py` — the in-memory ranking service. Runs ranking in a background thread, keeps the full ranked pool in `STATE`, and reuses **the exact** `score.finalize_ranking` so dashboard order matches `rank.py`. Provides leaderboard paging, grounded `_candidate_insights` (strengths/weaknesses/risk score/missing must-haves/similar roles), `analytics` (histograms, skills heatmap, funnel, tiers), `compliance` (disparate impact + bias flags + scoring explanation), `_classify_honeypot` (structured violation records), and `job_intent` (role confidence, signal-conflict warnings, retrieval metadata). - `nextai.py` — a provider-agnostic LLM chat assistant (OpenAI/Gemini/Anthropic via stdlib `urllib`) grounded **only** in a compact JSON snapshot of the live ranking; refuses to fabricate. - `supabase_store.py` — a dependency-free PostgREST client that persists each run (task + top-200 / shortlisted / honeypot rows + shortlists). Best-effort; never blocks the result.

web/ (Next.js 14 + Tailwind, port 3000): `app/page.tsx`, `app/layout.tsx`, `app/globals.css`, and components: `Leaderboard`, `CandidateDrawer`, `GovernanceView`, `CompareView`, `InsightsView`, `IntegrityView`, `PipelineView`, `RoleView`, `Controls`, `Sidebar`, `TopBar`, `Kpis`, `Logs`, `IngestBanner`, `Views`, `charts`, `icons`; plus `lib/api.ts` and the typed contract in `lib/types.ts`.

2.5 Tests, compliance assets, and committed artifacts

tests/ (the five-stage quality gates): - `test_format.py` — wraps the validator rules over the produced `submission.csv` (header, 100 rows, ID pattern, unique ranks, non-increasing scores + tie-break, varied reasoning). - `test_traps.py` — the decisive differentiator: a keyword-stuffer "HR Manager" must score *below* a plain-language engineer who built a recsys **even when the stuffer has higher semantic similarity**; a blatant honeypot must be floored to 0. - `test_disqualifiers.py` — each JD disqualifier gate tested in *both* directions (fires when it should, stays silent when it should not), plus the ghost-candidate gate, the off-list stuffer guard, and the nice-to-have / eval-rigor bonuses. - `test_metrics.py` — locks the metric-shaping invariants: scores strictly non-increasing, bands monotonic with rank, disjoint band sub-ranges, no band-0 profile in the top-10, deterministic ordering, and domain-anchored evidence ignoring generic management verbs.

compliance/ — `audit_trail/audit_*.json` (one immutable record per run) and `bias_audit_template.json`.

artifacts/ — the committed frozen index: `retriever.pkl.gz` (~1 MB) and `candidate_ids.pkl.gz` (row-alignment guard).

Other assets — `Dockerfile` + `.dockerignore` (CPU-only offline reproduction image), `requirements.txt` (pinned: `numpy==2.5.0`, `scipy`, `scikit-learn==1.9.0`, `orjson`, plus Streamlit/FastAPI/pandas), `requirements-embeddings.txt` (optional `sentence-transformers` + `torch`), `submission_metadata.yaml`, `supabase_schema.sql`, `.streamlit/config.toml`, and `honeypot_flag_report.md` (the audited 80-flag breakdown).

2.6 The two-phase architecture (precompute + inference) in detail

The pipeline is deliberately split into an **offline** phase that may exceed the time budget and an **online** phase that must not.

Phase A — OFFLINE precompute (`precompute.py` , ~5 min, network allowed):

```
candidates.jsonl
| load_candidates() (src/load.py)
▼
[build_document(c) for c in candidates] (src/features.py) — N text docs
| build_retriever(docs, backend) (src/retrieve.py)
▼
HybridRetriever.fit() → TF-IDF (N×V sparse) + LSA (N×256 dense) (~285s on 100K)
| retriever.retrieve(jd["query_text"]) ← replay for the FIXED JD query
▼
(shortlist[4000], dense_sim[N], lexical_sim[N])
| PrecomputedRetriever(...) (src/precomputed.py)
▼
gzip-pickle → artifacts/retriever.pkl.gz (~1 MB) + candidate_ids.pkl.gz
```

The crucial move: the full fitted matrices are ~220 MB (too big to commit), but `score_pool` only ever consumes the `shortlist` and `dense_sim` arrays. So `precompute.py` freezes **just those arrays** for the one fixed JD query — ~1 MB, deterministic from `RANDOM_SEED=13`, and committable for sandbox reproduction.

Phase B — ONLINE inference (`rank.py` , ~25 s, CPU, no network):

```
[0] HF_HUB_OFFLINE=1 / TRANSFORMERS_OFFLINE=1 (set before any ST import)
[1] load_jd() src/jd_intent.json
[0] load_candidates() src/load.py (~14s)
[2] _load_cached_retriever() rank.py → verifies dense.shape[0]==N,
    query_text match, AND candidate id/order match; else refit live (~0.3s load)
[3] score_pool(): retrieve → compute_features → integrity gate → council.deliberate
    → final_fit × availability + soft_nudge (~8s)
[3b] rerank() head of 200 src/rerank.py (<1s)
[9b] relevance_band + _assign_banded_scores src/score.py (<1s)
[10] reasoning.generate() × 100 src/reasoning.py
[7] fairness.audit() + [9] compliance.write_audit()
    write_csv() rank.py
```

The README's measured breakdown: cold (no index) ≈ **5.9 min** (retriever fit alone ≈ 285 s); with the committed index ≈ **25 s** (index load ≈ 0.3 s, scoring the 4000-shortlist ≈ 8 s).

`rank.py::_load_cached_retriever` guards correctness with three checks — row count, frozen `query_text`,

and exact `candidate_ids` order — and **refits live** on any mismatch (using the reduced `FAST_FIT_*` knobs so even the fallback stays under 5 minutes).

3. Why It Was Built That Way

3.1 Why this model over the alternatives

Why an interpretable ensemble instead of a trained LTR model (LambdaMART / neural ranker)? - No labels. The pool ships without graded relevance judgements, so a supervised pairwise/listwise loss cannot be trained honestly. The "ground truth" is the JD's stated priorities, which an interpretable rule ensemble encodes directly. - **Explainability is mandatory.** `compliance.py` classifies the system as "high-risk (Annex III, employment)" under the EU AI Act, which requires every decision-influencing weight to be documented and inspectable. A black-box model cannot satisfy `Art. 14` human-oversight or produce the grounded per-candidate reasoning the manual-review stage demands. Every weight lives in `src/config.py` and is echoed into the audit trail. - **Determinism + budget.** A heuristic ensemble is exactly reproducible (`RANDOM_SEED=13`) and microseconds-cheap per candidate, which keeps the whole pipeline inside the 5-minute CPU budget.

Why still add a neural cross-encoder (`rerank.py`) and bands (`score.py`)? A flat weighted sum gets the *set* right but loses fine *order*. Because **NDCG@10 is half the metric**, the design layers a pairwise cross-encoder on just the 200-deep head (where order matters most) and a listwise band gate over the whole list (to guarantee MAP/P@10 monotonicity) — recovering order-sensitivity without sacrificing interpretability or budget.

Why TF-IDF + LSA as the default retriever rather than embeddings? Embeddings need model weights (download + commit + load time) and risk a network call. TF-IDF + LSA is zero-download, offline, deterministic, and CPU-fast. Embeddings remain a **first-class, default-on upgrade** with automatic fallback (`build_retriever` , `STRetriever`), so the design gets neural quality *when available* and never breaks *when not*.

3.2 Why this feature set

The features in `features.py` are chosen to operationalise the JD's explicit value system, **evidence and identity over claims**:

- **Career descriptions are weighted 2× in `build_document`** because that is where genuine "built/shipped a system" evidence lives — the gold the JD asks for — whereas headers and skill lists are cheap to fake.
- **Evidence is domain-anchored** (`evidence_verbs` × `evidence_domain` in `jd_intent.json`): a delivery verb ("led", "owned") only counts when it co-occurs with an ML/system domain noun. This stops generic management language from inflating non-engineers — verified by `test_metrics.py::test_domain_anchored_evidence_ignores_generic_verbs` .
- **Skill-trust is verification-tempered, not raw counts** (`mask_piercer` , `relevant_trust`): each relevant skill is scaled by `0.5·duration + 0.3·endorsements + 0.2·assessment` . A wall of unendorsed, zero-duration "expert" skills earns almost nothing — the anti-keyword-stuffer core. The self-declared skills list is the *lowest-weighted* additive scorer (0.14) by design.

- **Title identity is explicit** (`name_rectifier` , `positive_titles` / `negative_titles`): an "HR Manager" current title scores 0.08; a genuine engineering title scores 1.0.
- **Disqualifier signals are multi-part** (`research_ratio` , `wrapper_skill_count` vs `core_ml_max_tenure` , `leadership_current_months` vs `github` , `has_external_validation`) so the JD's "we will not move forward" cases require *several* corroborating facts, never a single keyword.
- **Behavioural signals are first-class** (`days_inactive` , `response_rate` , `offer_acceptance` , engagement) because the JD warns about the "perfect on paper but not actually available" ghost.

The feature set is intentionally **pure-Python + datetime** (no heavyweight NLP) so it is fast and reproducible on CPU.

3.3 Why the two-phase architecture specifically — the bottleneck it solves

The single dominant cost in the pipeline is **fitting the hybrid retriever over 100K documents**: TF-IDF tokenisation + `TruncatedSVD` $\approx$ **285 s**, plus $\approx$ 15 s to build documents. That alone nearly exhausts the 5-minute budget and would have to be repeated on every run.

The two-phase split solves exactly this: - The expensive fit runs **once, offline** (where exceeding 5 minutes is allowed), and only its *result* for the fixed JD query is frozen. - The committed index is tiny (**~1 MB vs ~220 MB**) because only the consumed arrays (`shortlist` , `dense_sim`) are kept — see `precomputed.py` 's docstring. - `PrecomputedRetriever` lives in a **sklearn-free module on purpose**: unpickling it imports only NumPy, saving the ~13 s (more on a cold/AV Windows box) that dragging in scikit-learn just to deserialise would cost. - The result is byte-identical to a from-scratch run (same arrays in, same arrays out), so reviewers reproduce the CSV exactly while the online step finishes in ~25 s.

In short: the two-phase design converts a recurring ~6-minute, budget-threatening fit into a one-time offline cost plus a ~0.3 s load, **without** changing the output.

3.4 The explicit tradeoffs

Tradeoff	Decision in code	Rationale
Speed vs accuracy (retrieve r)	LSA default; ST embeddings opt-in via <code>REDROB_EMBED_BACKEND=st</code> ; large pools never live-encode (<code>ST_LIVE_MAX=20000</code>).	Guarantees the budget is never blown; neural quality is available offline via precompute.
Speed vs accuracy (re-rank)	<code>RERANK_BACKEND</code> defaults to <code>feature</code> , not <code>auto</code> / <code>cross-encoder</code> .	Weights aren't committed and the sandbox is offline, so an <code>auto</code> load would waste ~235 s on failed Hugging Face calls before falling back. The feature re-rank is deterministic, instant, and byte-identical. The neural path is enabled only where weights are cached.
Recall vs compute	<code>SHORTLIST_SIZE=4000</code> , <code>RERANK_SIZE=200</code> .	A deep shortlist protects recall (and NDCG@50/MAP); the expensive re-rank touches only the head, where NDCG@10 is decided.
Coverage vs reproducibility	<code>FAST_FIT_*</code> knobs (unigrams only, fewer SVD iters, smaller sample) for the cold live-fit fallback.	If the sandbox pool ever mismatches the committed index, the live refit (~150 s vs 285 s) still fits the budget — accepting a slight, documented deviation from the committed-index output for that edge case.
Strictness vs ranking	Only four HARD impossibility rules; the "skill duration > professional YOE"	Per <code>candidate_schema.json</code> , skill <code>duration_months</code> is <i>total</i> usage, distinct from professional tenure. Conflating them flagged 690 profiles (17.9% of the pool — over the 10% disqualification line) and

Tradeoff	Decision in code	Rationale
quality (integrity)	proxy is <i>deliberately removed</i> (<code>integrity.py</code> docstring).	excluded legitimate engineers. The corrected layer flags exactly 80 , matching the documented honeypot count (<code>honeypot_flag_report.md</code>).
Penalty vs erasure (gates)	<code>disqualifier_screen</code> floored at <code>DISQUAL_MULT_FLOOR=0.20</code> ; <code>neti_net</code> at <code>NEGSCREEN_MIN=0.40</code> ; availability bounded <code>[0.55, 1.10]</code> with a hard ghost gate $\times 0.55$.	A demoted-but-present elite candidate is better for ranking quality than an erased one; every gate needs <i>multiple</i> signals so no single keyword sinks a candidate (protects NDCG).
Boost vs dominance (bonuses)	Nice-to-have and eval-rigor bonuses are additive, outside the normalised core, capped at <code>0.05</code> / <code>0.04</code> .	JD nice-to-haves should "boost but never reject"; keeping them out of the core preserves the tested gem-over-stuffer ordering.
Simplicity vs realism (fairness)	Audit-and-log by default; the DELTR-style re-rank is available but off; gender is never inferred.	Avoids silently overriding genuine merit and avoids introducing a noisy, ethically fraught signal, while still measuring the 4/5ths rule.

4. How It Works — end-to-end walkthrough

This traces a single `python rank.py --candidates candidates.jsonl --out submission.csv` invocation, naming the exact functions and parameters involved.

Step 0 — Offline mode + JD load (`rank.py::main`)

Before any embedding backend can be imported, `rank.py` sets `HF_HUB_OFFLINE=1` and `TRANSFORMERS_OFFLINE=1` (module top) so an uncached model fails fast instead of retrying over the network. It then calls `load_jd()` (`score.py`), which reads `src/jd_intent.json` into a dict — the static JD interpretation that drives every downstream signal (`query_text` , `must_have_capabilities` , titles, evidence vocabularies, disqualifier lists).

Step 1 — Load candidates (`load.py::load_candidates`)

`load_candidates(path)` streams `candidates.jsonl` line by line with `orjson` (falling back to stdlib `json`), tolerating malformed lines, and returns a `list[dict]` of ~100,000 profiles. Each dict matches the schema in §A.1 (profile, career_history, skills, redrob_signals, ...).

Step 2 — Load or rebuild the retriever (`rank.py::_load_cached_retriever`)

`_load_cached_retriever(candidates, jd)` looks for `artifacts/retriever.pkl.gz` , unpickles the `PrecomputedRetriever` , and validates it three ways: 1. `retr.dense.shape[0] == len(candidates)` (row count), 2. `retr.query_text == jd["query_text"]` (the frozen query still matches), 3. `candidate_ids.pkl.gz` equals `[c["candidate_id"] for c in candidates]` (exact id/order — the frozen arrays are row-aligned).

If all pass, the index is used (load ≈ 0.3 s). On any mismatch it returns `None`, and `score_pool` refits a live `HybridRetriever` (with `FAST_FIT_*` knobs for large pools).

Step 3 — Retrieve the shortlist (`retrieve.py` / `precomputed.py`)

`score_pool` calls `retriever.retrieve(jd["query_text"])`: - The **live** path (`HybridRetriever.retrieve`) computes lexical cosine `tfidf @ q_tfidf.T` and dense cosine `dense @ q_dense.T`, both clipped to ≥ 0 , then fuses them with **Reciprocal Rank Fusion** (`RRF_K=60`) via a vectorised inverse-rank scatter, and returns the top `SHORTLIST_SIZE=4000` indices plus the full-length `dense_sim` / `lexical_sim` arrays. - The **cached** path (`PrecomputedRetriever.retrieve`) simply replays the frozen `(shortlist, dense_sim, lexical_sim)`.

The dense similarities for the shortlist are then min-max normalised to `[0,1]` (`sl_sims`, `lo`, `hi`, `rng` in `score_pool`) to produce the `sem` input for the Semantic Seer.

Step 4 — Feature engineering per candidate (`features.py::compute_features`)

For each shortlisted index, `compute_features(c, jd)` derives the full feature dict: `yoe`, `avg_tenure_months`, `product_ratio`, `services_only`, `title_pos` / `cur_title_pos` / `cur_title_neg`, **domain-anchored** `evidence_hits`, `relevant_trust` + `relevant_skill_count`, `offdomain_hits` / `irnlp_hits`, `days_inactive`, disqualifier signals (`research_ratio`, `wrapper_skill_count`, `core_ml_max_tenure`, `leadership_current_months`, `has_external_validation`), `nice_trust`, `eval_framework_hits`, and the behavioural block.

Step 5 — Integrity gate (`integrity.py::check`)

`integrity.check(c)` returns `(integrity_score, is_honeypot, reasons)`. If `is_honeypot` is `True` (any of the four HARD rules fired — e.g. ≥ 3 expert-with-0-months skills, or a skill predating its technology per `TECH_FIRST_YEAR`), the candidate is **excluded entirely** from scoring and counted in `n_honeypots` / `honeypot_rules`. Otherwise `integrity_score` (1.0, lightly reduced by soft penalties) becomes a multiplier.

Step 6 — Council deliberation (`council.py::deliberate`)

`deliberate(f, sem)` runs all nine scorers: - six additive scores (`semantic_seer`, `name_rectifier`, `evidence_scout`, `mask_piercer`, `path_reader`, `terrain_master`) fused by `COUNCIL_WEIGHTS` into `core =  $\sum(\text{part}_k \cdot w_k) / \sum w$` ; - three/four multipliers: `neg_mult` (`neti_net`), `avail_mult` (`availability_oracle`), and `disqualifier_mult` (`disqualifier_screen`); - plus a `rationales` dict of short, honest fragments for the reasoning layer.

Step 7 — Compose the pointwise score (`score.py::score_pool`)

```
final_fit = dec["core"] * integ[0] * dec["neg_mult"] * dec["disqualifier_mult"]
final      = final_fit * dec["avail_mult"] + _soft_nudge(f)
raw        = max(0.0, final)
```

`_soft_nudge(f)` adds bounded logistics + nice-to-have + eval-rigor bonuses (location, notice, `NICE_TO_HAVE_BONUS_MAX=0.05`, `EVAL_RIGOR_BONUS_MAX=0.04`). Each surviving candidate becomes a record `{idx, candidate_id, raw, f, dec, integ, candidate}`.

Step 8 — Two-stage head re-rank (`score.py::finalize_ranking` → `rerank.py::rerank`)

Records are sorted by `(-raw, candidate_id)` and each gets `order = raw`. The top `RERANK_SIZE=200` form the head. `rerank(jd, head)` returns a relevance score per head record — from the CPU cross-encoder if `RERANK_BACKEND` allows and weights are cached, else the deterministic `_feature_scores` (a sharpened blend `0.34·semantic + 0.30·evidence + 0.22·mask + 0.14·name`, gated, with bounded availability). The returned scores are min-max mapped **into the head's own raw range** so the head reorders internally but never crosses below the un-re-ranked tail.

Step 9 — Relevance bands + calibration (`score.py::relevance_band`, `_assign_banded_scores`)

Each record is assigned a band by `relevance_band(f, dec, integ)` : - **0 / WEAK** — strongly disqualified (`disqualifier_mult ≤ 0.5`), or a non-engineer title with no engineering history, or no relevant signal at all; - **2 / STRONG** — verified trust $\geq$ `BAND_STRONG_TRUST=1.5`, evidence $\geq$ `BAND_STRONG_EV=4`, engineering identity, not services-only, not gated; - **1 / STANDARD** — everything else.

Records are sorted by `(-band, -order, candidate_id)`, the top `TOP_N=100` are kept, and `_assign_banded_scores` maps each band's members by intra-band min-max into the **disjoint** ranges `BAND_RANGES = {2:(0.70,0.99), 1:(0.45,0.69), 0:(0.05,0.44)}`. A final sort by `(-score, candidate_id)` yields a globally non-increasing, tie-free list — satisfying the validator and the MAP/P@10/NDCG properties simultaneously.

Step 10 — Grounded reasoning (`reasoning.py::generate`)

For each of the top 100, `generate(c, f, dec, integ, score, rank, jd)` writes a 1–2 sentence justification: a lead clause (title + YOY, phrased by a `candidate_id`-seeded variant), the single strongest council driver, **only verified** skill names from `verified_relevant_skills`, an honest concern if one exists, and a confidence word (`_confidence_word`) calibrated to **both** score and rank band (so rank 95 never reads as "high-confidence fit").

Step 11 — Fairness audit, compliance trail, CSV (`rank.py`, `fairness.py`, `compliance.py`)

`fairness.audit(candidates, stats["selected_idx"])` computes the disparate-impact ratio for `region` and `institution_tier`. `compliance.write_audit(...)` writes an immutable `compliance/audit_trail/audit_<ts>.json` (input SHA-256 fingerprint, weights, honeypot counts by rule, timing, backend, fairness). Finally `write_csv(records, out)` emits `candidate_id,rank,score(4dp),reasoning`.

4.12 A worked contrast — why the gem beats the stuffer

The decisive case the JD itself describes (encoded in `tests/test_traps.py`):

- **The stuffer** — an "HR Manager" with every trendy AI skill listed, but **zero endorsements and zero usage**, and an HR career description. Even handed an adversarially high semantic similarity (`sem=0.95`), it loses: `name_rectifier` returns 0.08 (negative title), `mask_piercer` is capped (unverified), `neti_net`

A.2 Output sample (`submission.csv`)

```
candidate_id,rank,score,reasoning
CAND_0071974,1,0.9900,"Senior AI Engineer with 7.8 yrs; current title is a genuine engineering/ML
role; verified Qdrant, Embeddings, Elasticsearch. High-confidence fit."
CAND_0080766,2,0.9701,"Staff Machine Learning Engineer (8.8 yrs); ... verified Deep Learning,
Python, OpenSearch. High-confidence fit (concern: 90d notice)."
...
```

A.3 Key tunable knobs (`src/config.py`) — quick reference

Knob	Value	Role
<code>SHORTLIST_SIZE</code>	4000	retrieval recall depth
<code>RERANK_SIZE</code>	200	head size for the neural re-rank
<code>DENSE_DIM</code>	256	LSA dimensionality
<code>TFIDF_MAX_FEATURES</code> / <code>TFIDF_NGRAM</code>	50000 / (1,2)	lexical vocabulary
<code>RRF_K</code>	60	fusion constant
<code>COUNCIL_WEIGHTS</code>	evidence .22, name .20, semantic/terrain .16, mask .14, path .12	additive fusion
<code>AVAIL_MIN</code> / <code>AVAIL_MAX</code>	0.55 / 1.10	availability modifier bounds
<code>DISQUAL_MULT_FLOOR</code>	0.20	disqualifier floor
<code>HONEYPOT_EXPERT_ZERO_MIN</code>	3	impossibility threshold
<code>BAND_RANGES</code>	{2:(.70,.99), 1:(.45,.69), 0:(.05,.44)}	disjoint score tiers
<code>RANDOM_SEED</code> / <code>TOP_N</code>	13 / 100	determinism / output size

A.4 The dashboard data path (parallel to the submission path)

`api/ranker.py::_do_rank` mirrors `rank.py` but ranks the **whole** pool (not just 100) in a background thread, then reuses the exact `score.finalize_ranking` so the dashboard ordering matches the submission. The Next.js frontend (`web/`) polls `/api/status` and renders the leaderboard, candidate insights, analytics, governance/fairness, integrity log, role interpretation, pipeline shortlists (Supabase), and the NextAI assistant — all reading the same in-memory `STATE` produced by the Council of Nine.

End of report. Generated from a full read of the `redrob-ranker-v2/` source tree; every claim above is traceable to a named file, function, or configuration value in the repository.